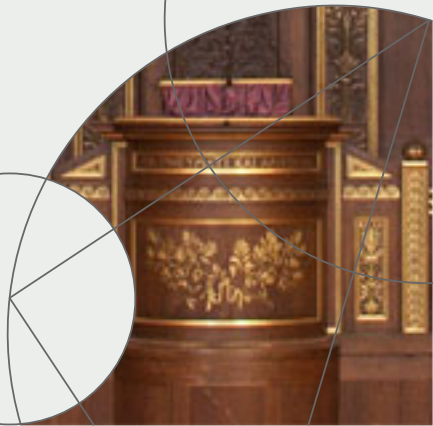
STEVENS INSTITUTE OF TECHNOLOGY

Data Acquisition and Processing I: Big Data

Rensheng Wang,
rwang1@stevens.edu
Dept. of Electrical and Computer Engineering
Stevens Institute of Technology



Recommender: 4-Stage Architecture

1. Retrieval - (Candidate Generation)
   - Goal:
     Narrow down from billions/millions of items → a few thousand/hundred of relevant candidates
2. Rough Ranking - (Filtering or Preliminary Ranking)
   - Goal: Refine the retrieved candidates to hundreds using lightweight ranking models or rules
3. Fine Ranking
   - Goal: Assign precise relevance scores to the filtered candidates
4. Re-ranking (Post-ranking)
   - Goal: Optimize the final ordering to dozens via considering business rules, diversity, fairness, and user experience,

Recommender: 4-Stage Architecture

1. Retrieval - (Candidate Generation)
   - Key ideas:
     Use fast, approximate models to retrieve likely relevant items.
     Usually uses embedding-based two-tower (dual encoder) models.
   - Methods:
     Content-based retrieval: match user/item embeddings (dot product or cosine similarity).
     Collaborative filtering: using user-item interaction matrix (matrix factorization, ANN search).
     Graph-based retrieval: e.g., random walks on user-item graph.
     Heuristic rules: popularity, trending, recency boosts.
   - Output: ~ 100-1000 candidate items per user.

Recommender: 4-Stage Architecture

☞ 2. Rough Ranking - (Filtering or Preliminary Ranking)

- Key ideas:
  Apply basic features such as recency, availability, or content safety.
  Can use shallow ML models (e.g., logistic regression, gradient boosting).
- Tasks:
  Remove duplicates, unavailable items, NSFW or policy-violating items.
  Basic scoring for click probability or engagement.
  Filter out irrelevant categories (based on context).
- Output:
  ~100-300 items remain.

Recommender: 4-Stage Architecture

☞ 3. Fine Ranking

- Key ideas:

  Use a deep model with rich features to maximize engagement metrics (CTR, watch time, dwell time, etc.).

  Models may include:

  - DNNs (e.g., YouTube's ranking network)
  - XGBoost/GBDT (for tabular data)
  - Contextual features (device, time, location, user history)
  - Multi-objective optimization (CTR, CVR, revenue, diversity)

- Output:

  Top-ranked ~ 20-50 items.

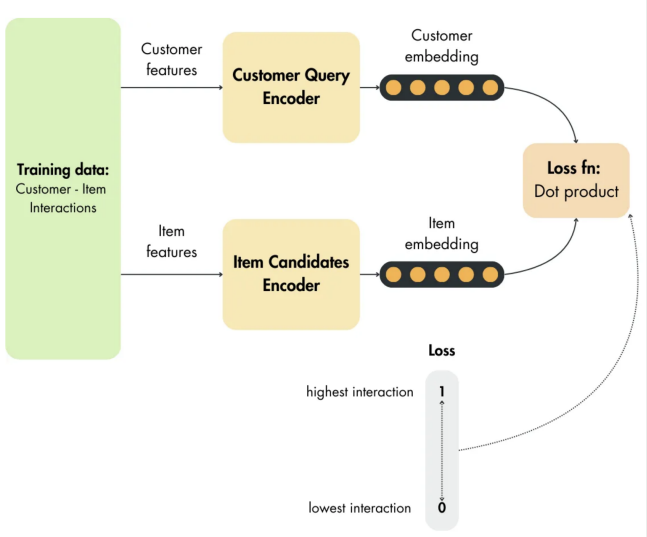
Recommender: 4-Stage Architecture

☞ 4. Re-ranking (Post-ranking)

- Key ideas:
  Adjust for diversity, novelty, serendipity, and exploration.
  Blend results from multiple sources (e.g., "Because you watched $\cdots$" + "Trending now").
  Apply multi-objective trade-offs (e.g., engagement vs. satisfaction vs. fairness).
- Methods:
  Re-ranking algorithms (e.g., MMR, xQuAD, fairness-aware ranking)
  Rule-based blending (content mix policies)
  Bandit or reinforcement learning to adaptively reorder.
- Output:
  Final ordered list shown to the user.

Tow Tower Model

- compute feature-rich embeddings for customers and items in the same embedding space
- calculate distance of customer's embeddings and items to search for the most relevant item candidates

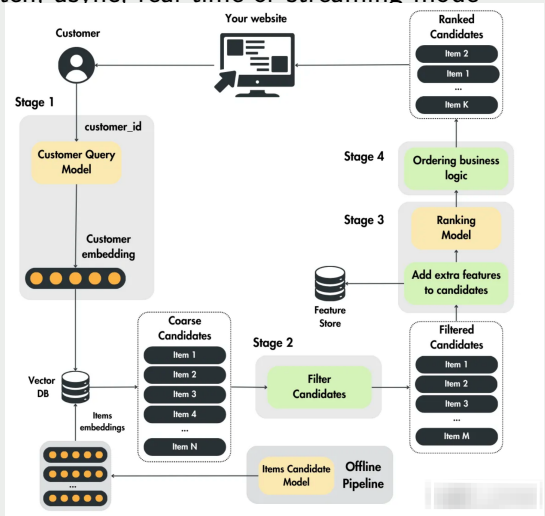


Tow Tower Model

- The two-tower model architecture trains two neural networks in parallel:
  - The customer query encoder transforms customer features into a dense embedding vector.
  - The item candidates encoder transforms item features into dense embeddings in the same vector space as the customer embeddings.
- Both encoders can process various types of features:
  - Customer encoder: demographic information, historical behavior, contextual features
  - Item encoder: tags, description, rating

4-Stage Architecture

- An **offline** pipeline that computes the candidate embeddings
- An **online** pipeline that computes the actual recommendations for a customer. This pipeline can run in batch, async, real-time or streaming mode



4-Stage Architecture

- Stage 1:

  This stage aims to process a large (>100M elements up to billions) corpus of candidate items and retrieve a relevant subset (hundreds) of items for downstream ranking and filtering tasks.

  The candidate generation step only provides broad personalization via collaborative filtering. Similarities are expressed in coarse features such as item and customer IDs.

  The pipeline takes a customer_id and other input features, such as the current date, computes the customer embedding using the Customer Query Model (trained using the Two Tower model), and queries the vector DB for similar candidate items.

  Using the customer's embedding, the vector DB (or index) scans the entire corpus and reduces it to xN potential candidates (~hundreds).

4-Stage Architecture

- Stage 2:

  Stage 2 takes the N candidate items and applies various filters, such as removing items already seen or purchased.

  The core idea is to filter out unnecessary candidates before proceeding to the most expensive operations from Stage 3. The filtering is often done using a Bloom filter, a space-efficient probabilistic data structure used to test whether an element is a set member (such as seen or purchased items).

  After this stage, we are left with only xM item candidates.

4-Stage Architecture

- Stage 3:

  Stage 3 takes the xM item candidates and prepares them for ranking. An algorithm that provides a score for each "(item candidate, customer)" tuple based on how relevant that item is to a particular customer.

  During ranking, we can access more features describing the item and the user's relationship, as only a few hundred items are being scored rather than the millions scored in candidate generation.

  The ranking step is slower as we enhance the items and customers with multiple features. We usually use a feature store to query all the necessary features.

  The ranking model can use a boosting tree, such as XGBoost or CatBoost, a neural network or even an LLM.

4-Stage Architecture

- Stage 3:

  After the ranking model scores each "(item candidate, customer)" tuple, we must order the items based on the ranking score plus other optional business logic.

  The highest-scoring items are presented to the user and ranked by their score.

  If the items candidate list is too extensive for our use case, we could further cut it to xK item candidates.

  It is critical to order the items based on relevance. Having the most personalized candidates at the top increases the customer's probability of clicking on them.

  For example, you want your No. 1 movie or playlist always to be the first thing when you open Netflix, YouTube or Spotify. You don't want to explore too much until you find it.